MMIS Dev Branch Audit Publication Index

Date: 2026-06-23

Publication target:

- ? Repository: `ERP-MIS/mmis-ura`
- ? Remote branch: `feat/mmis-dev-audit-publication-2026-06-23`
- ? Source workspace: `C:\FTS\NovoRepo_FOITS\DOWNLOADS\FTS-OITS\MMIS-URA`

Verdict

`PARTIALLY READY / BLOCKED FOR CLEAN UAT HANDOFF`

The backend dev branch has a real URA Partner Service foundation, route validation, rent-payment PRN support, EFRIS outbox backend surface, and passing local backend validation evidence. The frontend dev branch and API collections are behind that backend surface, so the system is not ready to present as a clean end-to-end UAT handoff for onboarding and fiscal operations.

Verified Branch / PR State

- ? GitHub PR: `oits-dhaka/MarketMasterApi#98`
- ? Title: `MMIS-420 rent payment PRN slice`
- ? State: `MERGED`
- ? Base: `dev`
- ? Head: `feature/mmis-420-prn-t109-handoff`
- ? Merge commit: `131ed32e94aa0f36c5d62ee7f17ff8f802215803`
- ? Merged at: `2026-06-22T10:32:29Z`

Current `MarketMasterApi` `origin/dev` already contains PR `#98`. The live residual backend difference captured in the branch-diff artifact is one feature hardening commit at `a84b916029280ea663fe5a8c127c675c1211808d` versus later dev delivery, webhook, and validation work.

Top P1 Findings

Area	Finding	Impact
Supplier signup	Frontend public signup does not send backend-required `taxId`	Supplier onboarding cannot cleanly satisfy the current backend schema
Vendor / onboarding KYC	Generic KYC step still has mock-only local submission behavior	UI can imply compliance persistence that does not exist
EFRIS operator tooling	Backend EFRIS outbox endpoints exist, but frontend admin tooling does not expose them	Operators cannot inspect or retry fiscalization queue state from the ERP UI
API collection parity	Canonical collection is incomplete against current backend dev route inventory	QA and UAT can miss live backend routes

Artifact Index

Artifact	Purpose
MMIS_DEV_BRANCH_DEEP_ANALYSIS_VERDICT_2026-06-23.md	Primary verdict report covering backend, frontend, onboarding, URA/EFRIS, InstantTIN, Hoppscotch/Postman status, and execution plan
MMIS_DEV_BRANCH_ENDPOINT_AND_COLLECTION_INVENTORY_2026-06-23.md	Backend endpoint inventory and collection-comparison narrative
MMIS_DEV_BRANCH_PAYLOAD_CONTRACT_RECHECK_2026-06-23.md	High-confidence request/response contract recheck
MMIS_DEV_BRANCH_ISSUE_MATRIX_AND_EXECUTION_PLAN_2026-06-23.md	Prioritized issue matrix, fix suggestions, and phased execution plan
MMIS_DEV_BRANCH_OBJECTIVE_COMPLETION_AUDIT_2026-06-23.md	Requirement-by-requirement proof that the requested audit objective was completed
MMIS_DEV_FEATURE_LOCAL_BRANCH_DIFF_TABLE_2026-06-23.md	Dev vs feature vs local workspace branch comparison
MMIS_DEV_BRANCH_API_COLLECTION_DIFF_2026-06-23.csv	Machine-readable route-vs-collection diff
MMIS_DEV_BRANCH_API_COVERAGE_MATRIX_2026-06-23.csv	Route-by-route collection coverage matrix
MMIS_DEV_BRANCH_AUDIT_PUBLICATION_PACK_2026-06-23.pdf	Consolidated reviewer PDF generated from the index and core verdict docs

Scope Boundary

This publication does not change source code in either product repository:

- ? `oits-dhaka/MarketMasterApi`
- ? `oits-dhaka/MarketMasterERP`

It publishes audit evidence and reviewer-facing documentation in `ERP-MIS/mmis-ura` only. The implementation gaps listed in the issue matrix remain separate follow-up work.

Recommended Next Execution Order

1. Fix supplier signup contract drift by adding `taxId` to frontend supplier signup.
2. Decide and wire supplier setup ownership.
3. Replace or remove mock-only KYC submit behavior from primary onboarding.
4. Expose backend EFRIS outbox in frontend admin tooling.
5. Regenerate the canonical API collection from current backend route inventory.

MMIS Dev Branch Deep Analysis And Verdict

Date: 2026-06-23

Scope:

- ? Backend dev branch: `MarketMasterApi` `origin/dev` at `f8a355d9d10d261a9bf1067b5e683de91659d955`
- ? Frontend dev branch: `MarketMasterERP` `origin/dev` at `008d31559644278593e3e15203a7ad5b2962bad1`
- ? Local API collection artifacts under `Docs/`
- ? Local URA, EFRIS, InstantTIN, smoke/UAT docs already present in this workspace

Source-of-truth rule used for this audit:

1. current backend and frontend dev worktrees under `.branch-work/.../_audit/...`
2. current backend/frontend route and service code
3. local collection artifacts and local runbooks
4. older narrative docs only when they still match current code

1. Executive Verdict

Verdict: `PARTIALLY READY / BLOCKED FOR CLEAN UAT HANDOFF`

Why:

- ? The backend dev branch is materially ahead of the collection/docs baseline and validates cleanly from this workspace for its URA unit tests and production bundle build.
- ? The frontend dev branch also builds successfully, but key onboarding and URA/EFRIS refinements are uneven: some flows are production-shaped, while other user-facing onboarding/KYC surfaces are still mock-driven or not wired to the backend contract that now exists.
- ? The main local backend Postman collection is a subset of the live backend route surface. It is not stale in the dangerous sense of documenting removed endpoints, but it is incomplete for current `dev`.
- ? URA Partner Service integration is real and code-backed. InstantTIN is still only a local evidence/SDK surface and is not integrated into runtime MMIS code.

Practical conclusion:

- ? `MarketMasterApi` `dev` is strong enough for continued backend URA/EFRIS delivery work.
- ? `MarketMasterERP` `dev` is not yet a fully coherent operational surface for all onboarding and URA/EFRIS user journeys.
- ? The current collections are usable for smoke and admin URA coverage, but not as a complete contract set for current backend `dev`.

2. Validation Performed

Backend `MarketMasterApi` `origin/dev`:

- ? `npm test` -> passed: `17 URA unit test(s) passed.`
- ? `npm run build` -> passed

Frontend `MarketMasterERP` `origin/dev`:

- ? `npm run build` -> passed after rerunning outside the sandbox
- ? Initial in-sandbox attempt failed with `Error: spawn EPERM`, which was environment-related rather than a code failure
- ? Production build completed, but Vite reported a large-chunk warning for `assets/index-CZ7Ve-Zp.js` at about `2.09 MB`

3. Backend Route Inventory

Deduped backend route surface found from current `MarketMasterApi` `origin/dev`: `192` endpoints.

Largest route groups by file:

Route file	Endpoint count	Notes
`src/routes/payment.routes.js`	40	rent, invoices, claims, refunds, webhook ops
`src/routes/ura.routes.js`	34	TIN, PRN, catalogs, audit, EFRIS outbox
`src/routes/auth.routes.js`	27	auth, verification, password, claim flows
`src/routes/product.routes.js`	12	mounted twice, creating alias paths
`src/routes/supplier.routes.js`	12	mounted twice, creating admin and superadmin aliases
`src/routes/vendor.routes.js`	12	mounted twice, creating onboarding and admin aliases

Important inventory nuance:

- ? `ura.routes.js` is mounted under both `/api/ura` and `/api/admin/ura`
- ? `product.routes.js` is mounted under both `/api/vendors` and `/api/products`
- ? `vendor.routes.js` and `supplier.routes.js` are also mounted under multiple prefixes

This is not automatically wrong, but it increases documentation drift risk and can make role boundaries less obvious in collections and client code.

4. Collection Comparison

Collection counts extracted locally:

Artifact	Endpoint count	Notes
backend code inventory	192	current `MarketMasterApi` `origin/dev`
`Docs/MMIS_Backend_Postman_Collection.json`	146	strongest local MMIS backend collection
`Docs/MMIS_Hoppscotch_Backend_Smoke_Cleaned.json`	57	smoke/UAT subset
`Docs/Market Master Api_for_sabit.json`	47	mixed/stale sample-state Hoppscotch export
`Docs/MMIS_URA_Provider_UAT_Only_Hoppscotch.json`	3	direct URA provider-side calls, not MMIS app routes

Normalized code-vs-collection comparison:

Comparison	Code endpoints	Collection endpoints	Code-only	Collection-only	Read
backend code vs `MMIS_Backend_Postman_Collection.json`	191 normalized	146 normalized	45	0	collection is a subset of code
backend code vs `MMIS_Hoppscotch_Backend_Smoke_Cleaned.json`	191 normalized	57 normalized	134	0	smoke pack is intentionally narrow

High-confidence interpretation:

? The main backend Postman collection does not invent routes that current code removed.

? The drift is one-directional: current backend `dev` has materially more live routes than the collection documents.

Largest missing categories from the main Postman collection:

1. delivery module endpoints under `/api/deliveries`
2. EFRIS outbox admin/runtime endpoints under `/api/ura/efris-outbox` and `/api/admin/ura/efris-outbox`
3. rent-payment PRN endpoints:

- `POST /api/ura/prn/rent-payment` - `POST /api/ura/rent-payments/:invoicePaymentId/prn` - admin aliases under `/api/admin/ura/...`

4. several alias surfaces created by duplicate route mounts:

- `/api/superadmin/suppliers...` - `/api/superadmin/vendors...` - `/api/products/:vendorId/products` - `/api/vendors/:productId`

Machine-readable diff export:

? `Docs/MMIS_DEV_BRANCH_API_COLLECTION_DIFF_2026-06-23.csv`

5. Onboarding / KYC / URA / InstantTIN Status

5.1 Vendor onboarding and KYC

Status: `PARTIAL, SPLIT BETWEEN REAL AND MOCKED SURFACES`

What is real:

? Backend `POST /api/vendors/setup-shop` is live and expects a multipart payload with:

- `marketId` - `shopName` - `monthlyRent` - `taxIdNumber` - `tinDocument` - optional `stallNumber`

? The backend verifies the TIN through `uraService.verifyTin(...)` before activating the shop flow.

? The frontend `components/onboarding/VendorShopSetup.tsx` is wired to that backend contract through `ApiService.vendors.setupShop(...)`.

What is still weak:

? `components/VendorKYCForm.tsx` is still a purely local document picker with a `setTimeout` mock submit and no backend upload or persistence.

? `components/onboarding/OnboardingWizard.tsx` still uses that mock KYC component for the generic identity step, which makes the overall onboarding story inconsistent.

Meaning:

? The vendor shop/TIN setup path is real.

? The broader FE onboarding/KYC UX is only partially refactored and still contains placeholder behavior.

5.2 Supplier onboarding and KYC

Status: `BACKEND READY, FRONTEND PARTIALLY WIRED, PUBLIC SIGNUP BROKEN`

What is real:

? Backend supplier signup schema now requires `taxId`.

? Backend supplier profile setup endpoint requires multipart `businessName`, `businessType`, `taxIdNumber`, and `tinDocument`.

? Backend supplier create/update flows can trigger `uraService.verifyTin(...)`.

What is missing or broken:

? Frontend public signup in `components/auth/AuthPage.tsx` calls `ApiService.auth.registerSupplier(...)` without sending `taxId`.

? Current FE signup form state does not expose a `taxId` field at all.

? FE has an API helper for multipart supplier setup (`ApiService.vendors.setupSupplier`), but no discovered current UI consumer for it in the dev worktree.

Meaning:

? Supplier onboarding refactoring is incomplete.

? The backend contract has moved ahead of the current FE public supplier signup surface.

5.3 Approval gating for vendor/supplier roles

Status: `IMPLEMENTED ON BACKEND, PARTIALLY ENFORCED IN FE`

Implemented backend behavior:

? Vendor/supplier approval is blocked unless the latest TIN verification status is `VERIFIED` or `OVERRIDDEN`.

? This gate exists in backend approval logic and in URA helper service checks.

Implemented frontend behavior:

? Pending approvals UI uses `TinVerificationBadge` and disables approval actions when TIN status is not approval-ready.

? Vendor and supplier data grids now expose tax-compliance and latest-verification context.

Meaning:

? The compliance rule is now real, not just visual.

? However, some onboarding entry flows still do not collect the backend-required data early enough.

5.4 URA Partner Service

Status: `IMPLEMENTED`

Confirmed live backend surfaces:

? health

? TIN verification

? PRN detail lookup

? generic PRN generation

? rent-payment PRN generation

? MDA catalog lookup

? MDA tax-code lookup

? tax-head detail

? request audit list/detail/retry

? TIN verification history and override

? EFRIS outbox list, enqueue, retry

Frontend status:

? Admin URA console exists and covers status, TIN, PRN, catalog, and request audit tabs.

? FE client includes URA health, verify, generic PRN, catalog, audit request, and TIN override methods.

? FE client does not currently expose the backend EFRIS outbox endpoints.

? FE docs explicitly note that no frontend EFRIS queue screen exists yet.

5.5 URA route validation

Status: `IMPLEMENTED`

Confirmed:

? shared validation middleware supports segmented `body`, `params`, and `query`

? `ura.validation.js` defines segmented schemas for most URA operations

? URA route group uses that middleware consistently for verification, PRN, audit, and outbox endpoints

This is one of the stronger refinements in current backend `dev`.

5.6 InstantTIN API / SDK

Status: `NOT INTEGRATED INTO MMIS RUNTIME`

Confirmed:

? local InstantTIN SDK and docs exist in the workspace

? local InstantTIN endpoint and auth evidence reports are present

? no current backend runtime code path in `MarketMasterApi` `origin/dev` imports or wraps the InstantTIN SDK

? current MMIS TIN verification path is still based on URA Partner Service `get-tin-details`

Meaning:

? InstantTIN is a researched candidate integration family, not a shipped MMIS integration in current `dev`

6. Request / Response Model Recheck

Confidence: `VERY HIGH` for the following contracts because they are derived directly from current route, validation, controller, and FE client code.

6.1 `POST /api/auth/register-supplier`

Expected backend body:

```
{
  "name": "string",
  "email": "user@example.com",
  "password": "min-8-char-password",
  "businessName": "string",
  "marketId": "uuid-or-market-id",
  "taxId": "URA TIN",
  "supplierType": "WHOLESALER|DISTRIBUTOR|MANUFACTURER|IMPORTER|LOCAL_PRODUCER",
  "licenseNumber": "optional",
  "warehouseAddress": "optional",
  "deliveryRadius": 10,
  "minimumOrder": 50000,
  "paymentTerms": "optional"
}
```

Current FE request body is incomplete because it does not send `taxId`.

6.2 `POST /api/vendors/setup-shop`

Expected backend request type: `multipart/form-data`

Required fields:

- ? `marketId`
- ? `shopName`
- ? `monthlyRent`
- ? `taxIdNumber`
- ? `tinDocument`

Optional:

- ? `stallNumber`

Current FE vendor shop setup appears aligned to this contract.

6.3 `POST /api/suppliers/setup-profile`

Expected backend request type: `multipart/form-data`

Required fields:

- ? `businessName`
- ? `taxIdNumber`
- ? `tinDocument`

Optional/currently accepted:

- ? `businessType`

Current FE has an API helper for this payload, but no discovered current user-facing screen that drives it.

6.4 `POST /api/ura/tin/verify`

Expected JSON body:

```
{
  "tin": "string",
  "submittedName": "optional string",
  "entityType": "VENDOR|SUPPLIER",
  "vendorId": "optional uuid",
  "supplierId": "optional uuid",
  "stakeholderId": "optional uuid"
}
```

Expected success envelope:

```
{
  "success": true,
  "message": "TIN verification completed",
  "data": {
    "id": "uuid",
    "status": "VERIFIED|MISMATCH|INVALID|FAILED|OVERRIDDEN|PENDING",
    "normalizedTin": "string"
  }
}
```

6.5 `POST /api/ura/prn/generate`

Expected JSON body:

```
{
  "amount": 150000,
  "tin": "string",
  "taxHead": "string",
  "taxPayerName": "optional string",
  "assessmentDate": "optional string",
  "paymentType": "optional string",
```

```
  "referenceNo": "optional string",
  "paymentPeriod": "optional string",
  "expiryDays": 30,
  "vendorId": "optional uuid",
  "supplierId": "optional uuid",
  "stakeholderId": "optional uuid",
  "marketId": "optional uuid",
  "invoiceId": "optional uuid",
  "taxPaymentId": "optional uuid"
}
```

6.6 Error model

Segmented validation failures are consistently shaped as:

```
{
  "success": false,
  "message": "Validation error",
  "errors": [
    {
      "path": "body.tin",
      "field": "tin",
      "message": "TIN is required",
      "code": "too_small"
    }
  ]
}
```

7. Issues And Fix Suggestions

Priority	Area	Issue	Evidence	Suggested fix
P1	FE signup	Supplier signup UI does not send required `taxId`	FE `AuthPage.tsx` omits it; BE `auth.validation.js` requires it	add `taxId` to the supplier signup schema, form state, UI, and request payload
P1	FE onboarding	Generic KYC component is still mock-only	`VendorKYCForm.tsx` uses local file state and `setTimeout`	replace with real multipart upload/persist flow or remove it from primary onboarding until implemented
P1	FE parity	No discovered UI consumer for multipart supplier setup	FE API helper exists; no current screen calls it	add a supplier setup screen after email verification/login or collapse that contract into signup if intended
P1	FE URA/EFRIS parity	FE URA client lacks EFRIS outbox methods and screen coverage	backend has outbox routes; FE docs explicitly say queue screen is absent	add FE API methods and an admin outbox queue view before claiming EFRIS operator readiness
P1	Collection drift	Main backend Postman collection is missing 45	code inventory 191 normalized vs collection 146	regenerate collection from current routes and tag missing

Priority	Area	Issue	Evidence	Suggested fix
		normalized current-code routes		route groups by module
P2	Route alias sprawl	Duplicate mounts create multiple equivalent endpoint families	`/api/ura` and `/api/admin/ura`; `/api/vendors` and `/api/products`	decide which aliases are canonical and document/deprecate the rest
P2	FE bundle size	Frontend production bundle emits a large-chunk warning	Vite build warning on `index` bundle ~2.09 MB	split dashboard/admin surfaces with dynamic imports and review heavy module boundaries
P2	UAT surface clarity	`Market Master Api_for_sabit.json` contains hardcoded sample state and is not a clean contract artifact	real IDs/tokens and placeholder endpoint gaps	archive it as legacy/sample-only and stop treating it as a source-of-truth collection
P3	InstantTIN delivery	InstantTIN evidence is present but runtime integration is absent	local SDK/docs only, no runtime imports	keep InstantTIN status explicitly marked “research/integration candidate” until a real adapter exists

8. Hoppscotch / Postman Testing Status

Current local testing assets:

- ? `Docs/MMIS_Backend_Postman_Collection.json`
- ? `Docs/MMIS_Hoppscotch_Backend_Smoke_Cleaned.json`
- ? `Docs/MMIS_URA_Provider_UAT_Only_Hoppscotch.json`
- ? `Docs/LOCAL_SMOKE_UAT_RUNBOOK.md`

High-confidence procedure from current workspace:

1. Start backend in `mock` mode for local smoke.
2. Use the MMIS backend Postman collection for broader app API coverage.
3. Use the cleaned Hoppscotch smoke set for a narrower happy-path pass.
4. Use the URA-provider Hoppscotch set only for direct URA gateway verification, not as proof of MMIS app-route readiness.
5. For sandbox/UAT, follow the existing runbook and keep frontend config proxy-only; do not expose URA secrets in FE config.

Important limitation:

? Hoppscotch assets do not yet cover the full current backend route surface, especially delivery and EFRIS-outbox additions.

9. Overall Plan Of Execution

1. Fix FE supplier signup contract drift first.

- add `taxId` UI/state/validation - verify successful signup against current BE schema

2. Unify onboarding/KYC surfaces.

- replace mock-only KYC submission behavior - explicitly choose between “signup-time data capture” and “post-login setup flow”

3. Expose backend EFRIS outbox parity in FE admin tools.

- add FE API methods - add queue list/retry surface

4. Regenerate the canonical backend API collection from current `dev`.

- include delivery, outbox, rent-payment PRN, and admin alias decisions

5. Decide and document canonical endpoints.

- especially for URA/admin URA and product/vendor duplicate mounts

6. Keep InstantTIN out of “implemented” status until a real MMIS adapter exists.

- if it is a roadmap item, define explicit ticketed scope instead of leaving it implied by docs

10. Bottom Line

Backend `dev` is ahead of frontend and collections in a useful way: the URA Partner Service, approval gates, route validation, and EFRIS outbox skeleton are real. The frontend `dev` branch has meaningful URA progress, but onboarding and operator parity are still inconsistent. The strongest immediate corrective action is not new backend work; it is closing the FE contract gaps and regenerating the canonical API collection from the current backend route inventory.

MMIS Dev Branch Issue Matrix And Execution Plan

Date: 2026-06-23

Related artifacts:

? MMIS_DEV_BRANCH_DEEP_ANALYSIS_VERDICT_2026-06-23.md

? MMIS_DEV_BRANCH_PAYLOAD_CONTRACT_RECHECK_2026-06-23.md

? MMIS_DEV_BRANCH_ENDPOINT_AND_COLLECTION_INVENTORY_2026-06-23.md

? MMIS_DEV_BRANCH_API_COLLECTION_DIFF_2026-06-23.csv

? MMIS_DEV_BRANCH_API_COVERAGE_MATRIX_2026-06-23.csv

? MMIS_DEV_FEATURE_LOCAL_BRANCH_DIFF_TABLE_2026-06-23.md

1. Release Readiness Summary

Overall verdict: `PARTIALLY READY / BLOCKED FOR CLEAN UAT HANDOFF`

Current readiness by surface:

Surface	Status	Read
Backend URA partner-service flows	ready for continued delivery	route, validation, controller, and test-backed
Backend EFRIS outbox slice	backend-ready, frontend-absent	queue and retry exist in API only
Vendor onboarding	partially ready	real shop + TIN setup exists, generic KYC UX is still mock-shaped
Supplier onboarding	not coherent end to end	BE requires tax data, FE public signup does not collect it
FE admin URA tooling	partially ready	TIN, PRN, catalog, audit tabs exist; outbox does not
API collections	incomplete	main collection lags current backend dev
InstantTIN integration	not implemented	docs and SDK present, runtime adapter absent

2. Prioritized Issue Matrix

Priority	Area	Current state	Evidence	Risk	Fix direction	Validation gate
P1	Supplier public signup	FE request omits required `taxId` while BE schema requires it	`AuthPage.tsx` request shape vs backend `auth.validation.js`; documented in payload recheck	supplier self-onboarding fails or cannot satisfy backend contract	add `taxId` field to FE state, form UI, client request, and any FE-side validation	successful `POST /api/auth/register-supplier` with current backend schema and no ad hoc payload patching

Priority	Area	Current state	Evidence	Risk	Fix direction	Validation gate
P1	Supplier setup flow	multipart setup helper exists but no confirmed current UI consumer was found	FE helper exists in `services/api.ts`; no discovered screen driver	supplier cannot complete tax/TIN setup from the intended UX	create explicit post-login supplier setup screen or collapse required data into signup	user can complete signup and then reach a real `setup-profile` screen that persists TIN document and backend status
P1	Vendor KYC consistency	generic KYC step still uses local-only mock submit behavior	`VendorKYCForm.tsx` uses local state and `setTimeout`; verdict doc records this	misleading onboarding state, no persisted compliance evidence	replace with real upload/persist flow or remove from primary onboarding until backend contract exists	document upload reaches backend and returned compliance state is reflected in the UI
P1	FE EFRIS parity	backend EFRIS outbox exists, FE has no client methods or queue screen	backend routes in `ura.routes.js`; FE changelog and audit show no outbox support	operators cannot inspect or retry fiscalization queue items	add FE URA client methods for outbox list/enqueue/retry and an admin queue surface	queue items visible in FE and retry action reaches backend
P1	Canonical API collection drift	main Postman collection covers 146 normalized routes vs 191 in current code	collection diff CSV and endpoint inventory report	QA/UAT operates against incomplete contract set	regenerate canonical collection from live route inventory and choose canonical aliases	exported collection includes delivery, rent-payment PRN, EFRIS outbox, and chosen admin aliases
P2	Endpoint alias sprawl	same functionality is exposed under multiple mounted families	`/api/ura` + `/api/admin/ura`, `/api/vendors` + `/api/products`, superadmin aliases	client confusion, doc drift, auth-boundary ambiguity	declare canonical public/admin paths and deprecate or explicitly document the rest	route catalog annotated with canonical vs compatibility aliases
P2	FE bundle size	production build completes with large	`npm run build` warning on	slower operator UI, harder future FE scaling	split heavy admin/dashboard surfaces using dynamic	FE build still passes and main chunk

Priority	Area	Current state	Evidence	Risk	Fix direction	Validation gate
		main bundle warning	~2.09 MB main chunk		imports and review shared module boundaries	size drops materially
P2	Legacy sample collection usage	`Market Master Api_for_sabit.json` is not a clean source-of-truth contract artifact	inventory report classifies it as mixed/legacy sample export	stale IDs and sample assumptions leak into testing practice	archive it as legacy-only and stop using it as canonical contract evidence	README or docs explicitly mark it sample-only
P2	Rent-payment PRN FE parity	backend rent-payment PRN endpoints exist, FE does not expose them	payload recheck shows backend-only status	rent PRN generation remains admin/backend-only despite backend support	add FE helper and operator flow only if product intent requires it	operator can generate rent-payment PRN from intended UI surface
P3	InstantTIN scope ambiguity	local docs and SDK exist but runtime integration is absent	audit verdict, local InstantTIN docs, no runtime imports	project status can be overstated in demos or docs	keep status explicitly marked research-only until adapter, config, and tests exist	runtime code imports adapter and smoke/UAT evidence exists before status changes

3. Strongest Code-Review Findings

These are the highest-signal findings from the current backend and frontend dev worktrees.

Severity	Finding	Why it matters
P1	FE supplier signup is contract-broken against the current backend	current user-facing flow cannot satisfy required compliance inputs
P1	Onboarding UX mixes real URA-backed TIN verification with mock-only KYC behavior	operator confidence and auditability are weaker than the UI implies
P1	Backend EFRIS surface is ahead of frontend/admin operational tooling	the system is harder to operate and verify end to end
P1	Canonical collection artifacts no longer represent current backend dev	test coverage and stakeholder expectations drift away from live code
P2	Route aliases multiply equivalent endpoints	parity work, documentation, and client ownership become harder over time

Severity	Finding	Why it matters
P2	FE delivery of new backend capabilities is inconsistent	backend progress is not automatically meaningful to operators or onboarding users

4. Execution Plan

Phase 1: Close Blocking FE/BE Contract Gaps

Goal: Make supplier and vendor onboarding contracts coherent enough for clean smoke/UAT.

Work:

1. add `taxId` to supplier public signup FE flow
2. decide whether supplier tax/TIN capture is signup-time or post-login setup-time
3. replace or temporarily remove mock-only KYC submit behavior from the primary onboarding wizard
4. verify that returned backend TIN/compliance state is actually rendered after submission

Exit criteria:

? supplier signup no longer sends a payload the backend rejects by schema

? no primary onboarding screen implies persistence when only local mock state exists

Phase 2: Expose Backend-Ready Operator Surfaces

Goal: Bring frontend admin capability closer to the actual backend URA/EFRIS feature set.

Work:

1. add FE client methods for EFRIS outbox list, enqueue, and retry
2. add an admin queue screen or tab for EFRIS outbox operations
3. decide whether rent-payment PRN generation must be exposed in FE now or remain backend/admin-only

Exit criteria:

? operator can inspect backend queue state from FE

? retry path can be exercised without direct API tooling

Phase 3: Regenerate Canonical Contract Artifacts

Goal: Make collections and route documentation match current backend dev.

Work:

1. use current route inventory as the canonical source
2. regenerate the main Postman collection for live backend `dev`
3. annotate or drop legacy/sample collections that are no longer contract-grade
4. explicitly mark canonical vs compatibility aliases

Exit criteria:

? main collection no longer misses delivery, EFRIS outbox, and rent-payment PRN routes

? testing packs are clearly labeled as full-contract vs smoke-only

Phase 4: Quality And Maintainability Pass

Goal: Reduce drift and improve confidence before wider rollout.

Work:

1. review route alias policy and reduce ambiguity where feasible
2. trim FE bundle weight on admin-heavy surfaces

3. keep InstantTIN status explicitly non-runtime until a real adapter, config path, and tests exist

Exit criteria:

? route ownership and canonical paths are documented

? FE build still passes after chunk-splitting work

? docs no longer imply shipped InstantTIN support

5. Minimal Verification Matrix

These are the smallest credible checks that should prove the fixes rather than just suggest them.

Area	Minimum credible check
Backend build health	`npm run build` in backend dev worktree
Backend URA regression	`npm test` in backend dev worktree with URA tests passing
FE release health	`npm run build` in frontend dev worktree
Supplier signup contract	exercise FE request shape against current backend schema and confirm `taxId` presence
Supplier setup flow	confirm a real FE screen drives `multipart/form-data` to `POST /api/suppliers/setup-profile`
Vendor KYC	confirm any remaining KYC step persists to backend or is clearly marked as not yet implemented
EFRIS outbox FE parity	verify queue list and retry actions call backend endpoints successfully
Collection parity	compare regenerated collection against current backend inventory and show zero collection-only stale routes

6. Recommended Order Of Work

1. fix supplier signup contract drift
2. decide and wire supplier setup ownership
3. remove or replace mock-only onboarding KYC behavior
4. expose EFRIS outbox in FE admin tooling
5. regenerate canonical collections from live backend routes
6. document canonical endpoint families
7. do bundle-size cleanup and non-blocking maintainability work

7. Final Recommendation

Do not present the current dev branches as fully UAT-ready for onboarding and fiscal operations.

A defensible status line is:

`backend URA/EFRIS foundation is real and test-backed; frontend onboarding parity and operator tooling still need targeted completion before a clean end-to-end UAT handoff`

MMIS Dev Branch Payload Contract Recheck

Date: 2026-06-23

Confidence: `VERY HIGH`

Why:

? derived from current backend `origin/dev` route files, validation schemas, controllers, and FE API-client usage

? not inferred from older screenshots or narrative docs

1. Supplier Public Signup

Backend route:

? `POST /api/auth/register-supplier`

Backend contract from `src/validations/auth.validation.js`:

```
{
  "name": "string",
  "email": "user@example.com",
  "password": "minimum 8 chars",
  "businessName": "string",
  "marketId": "string",
  "taxId": "URA TIN",
  "supplierType": "WHOLESALER|DISTRIBUTOR|MANUFACTURER|IMPORTER|LOCAL_PRODUCER",
  "licenseNumber": "optional",
  "warehouseAddress": "optional",
  "deliveryRadius": 10,
  "minimumOrder": 50000,
  "paymentTerms": "optional"
}
```

Observed FE request from `components/auth/AuthPage.tsx`:

```
{
  "name": "string",
  "email": "string",
  "password": "string",
  "phoneNumber": "string",
  "marketId": "string",
  "role": "Supplier",
  "supplierType": "string",
  "businessName": "string"
}
```

Alignment verdict:

? `supplierType` and `businessName` are aligned

? `taxId` is missing from FE

? `phoneNumber` and `role` are sent by FE but not part of the backend-required schema

Status:

? `BROKEN CONTRACT`

2. Vendor Shop Setup

Backend route:

? `POST /api/vendors/setup-shop`

Request type:

? `multipart/form-data`

Backend required fields:

? `marketId`

? `shopName`

? `monthlyRent`

? `taxIdNumber`

? `tinDocument`

Backend optional fields:

? `stallNumber`

Observed FE request from `services/api.ts` and `components/onboarding/VendorShopSetup.tsx`:

? uses `FormData`

? sends `marketId`

? sends `shopName`

? sends `monthlyRent`

? sends `taxIdNumber`

? sends `tinDocument`

? optionally sends `stallNumber`

Alignment verdict:

? `ALIGNED`

Expected success envelope:

```
{
  "success": true,
  "message": "Shop and stall setup completed successfully",
  "data": {
    "shop": {},
    "stall": {},
    "user": {
      "id": "string",
      "email": "string",
      "role": "Vendor",
      "vendorId": "uuid",
      "kycStatus": "VERIFIED",
      "taxComplianceStatus": "TIN_VERIFIED"
    },
    "tinVerification": {
      "status": "VERIFIED|OVERRIDDEN"
    }
  }
}
```

Expected failure modes:

? missing `taxIdNumber`

? missing `tinDocument`

? failed URA TIN verification

3. Supplier Profile Setup

Backend route:

? `POST /api/suppliers/setup-profile`

Request type:

? `multipart/form-data`

Backend required fields:

? `businessName`

? `taxIdNumber`

? `tinDocument`

Backend optional fields:

? `businessType`

Observed FE API helper:

? `ApiService.vendors.setupSupplier(...)`

Observed FE screen consumer:

? no current confirmed dev UI consumer found in this pass

Alignment verdict:

? helper contract exists

? user-facing flow not clearly wired

Status:

? `PARTIALLY WIRED`

4. URA TIN Verification

Backend route:

? `POST /api/ura/tin/verify`

Validated backend body:

```
{
  "tin": "string",
  "submittedName": "optional string",
  "entityType": "VENDOR|SUPPLIER",
  "vendorId": "optional uuid",
  "supplierId": "optional uuid",
  "stakeholderId": "optional uuid"
}
```

Observed FE request:

? `ApiService.ura.verifyTin(...)` matches this shape

Alignment verdict:

? `ALIGNED`

Expected success envelope:

```
{
  "success": true,
  "message": "TIN verification completed",
  "data": {
    "id": "uuid",
    "tin": "string",
    "normalizedTin": "string",
    "status": "PENDING|VERIFIED|MISMATCH|INVALID|FAILED|EXPIRED|OVERRIDDEN",
    "submittedName": "optional string",
    "uraTaxpayerName": "optional string",
    "failureReason": "optional string"
  }
}
```

5. URA Generic PRN Generation

Backend route:

? `POST /api/ura/prn/generate`

Validated backend body:

```
{
  "amount": 150000,
  "tin": "string",
  "taxHead": "string",
  "taxPayerName": "optional string",
  "assessmentDate": "optional string",
  "paymentType": "optional string",
  "referenceNo": "optional string",
  "paymentPeriod": "optional string",
  "expiryDays": 30,
  "vendorId": "optional uuid",
  "supplierId": "optional uuid",
  "stakeholderId": "optional uuid",
  "marketId": "optional uuid",
  "invoiceId": "optional uuid",
  "taxPaymentId": "optional uuid"
}
```

Observed FE request:

? `ApiService.ura.generatePrn(...)` matches the current backend shape closely

Alignment verdict:

? `ALIGNED`

6. URA Rent-Payment PRN Generation

Backend routes:

? `POST /api/ura/prn/rent-payment`

? `POST /api/ura/rent-payments/:invoicePaymentId/prn`

Validated backend schema in current dev:

```
{
  "invoicePaymentId": "uuid",
  "force": true
}
```

Important nuance:

? current `rentPaymentPrnSchema` is effectively a flat object with `invoicePaymentId` and `force`

? the route `/rent-payments/:invoicePaymentId/prn` reads from `req.validated.invoicePaymentId`, which works for the flat schema but is not expressed as a segmented `params` object

? this is functional in current tests, but not as structurally explicit as the other URA schemas

Observed FE support:

? no current FE API helper or admin UI action discovered for this specific rent-payment PRN route

Status:

? `BACKEND IMPLEMENTED / FE NOT EXPOSED`

7. URA Audit And Override Flows

Routes and alignment:

Route	Backend	FE client	Status
`GET /admin/ura/requests`	yes	yes	aligned
`GET /admin/ura/requests/:id`	yes	yes	aligned
`POST /admin/ura/requests/:id/retry`	yes	yes	aligned
`GET /admin/ura/tin-verifications`	yes	yes	aligned
`POST /admin/ura/tin-verifications/:id/override`	yes	yes	aligned

8. EFRIS Outbox

Backend routes:

- ? `GET /api/ura/efris-outbox`
- ? `POST /api/ura/efris-outbox/enqueue-from-request`
- ? `POST /api/ura/efris-outbox/:id/retry`
- ? admin aliases under `/api/admin/ura/...`

Backend validated request bodies:

Enqueue:

```
{
  "uraServiceRequestId": "uuid",
  "workflowType": "INVOICE_SUBMISSION",
  "payload": {},
  "metadata": {}
}
```

Retry:

```
{
  "reason": "minimum 5 chars"
}
```

Observed FE support:

- ? no current FE API methods for EFRIS outbox
- ? no current admin queue screen in current dev

Status:

- ? `BACKEND ONLY`

9. Validation Error Shape

Current backend validation middleware returns:

```
{
  "success": false,
```

```
  "message": "Validation error",
  "errors": [
    {
      "path": "body.tin",
      "field": "tin",
      "message": "TIN is required",
      "code": "too_small"
    }
  ]
}
```

This is one of the more consistent contracts in current backend dev and should be preserved as the canonical validation error model.

10. Contract Verdict

Flow	Verdict
vendor shop setup	aligned
URA TIN verify	aligned
URA generic PRN	aligned
URA audit/override	aligned
supplier public signup	broken FE/BE contract
supplier multipart setup	helper exists, UI parity incomplete
rent-payment PRN	backend only in current dev
EFRIS outbox	backend only in current dev

MMIS Dev Branch Endpoint And Collection Inventory

Date: 2026-06-23

Code source:

? `MarketMasterApi` `origin/dev` at `f8a355d9d10d261a9bf1067b5e683de91659d955`

Collection source files:

? `Docs/MMIS_Backend_Postman_Collection.json`

? `Docs/MMIS_Hoppscotch_Backend_Smoke_Cleaned.json`

? `Docs/Market Master Api_for_sabit.json`

? `Docs/MMIS_URA_Provider_UAT_Only_Hoppscotch.json`

Machine-readable supporting files:

? backend-dev-endpoints.json

? backend-collection-comparison.json

? MMIS_DEV_BRANCH_API_COLLECTION_DIFF_2026-06-23.csv

1. Inventory Summary

Surface	Endpoint count	Meaning
backend code inventory	192	deduped routes discovered from current backend dev route mounts
main Postman collection	146	strongest local MMIS app-route collection
cleaned Hoppscotch smoke	57	smoke subset only
`Market Master Api_for_sabit.json`	47	mixed/legacy hoppscotch export
URA provider-only Hoppscotch	3	direct URA provider calls, not MMIS app routes

2. Backend Route Groups

Grouped by mounted path family:

Route group	Count	Notes
`/api/vendors`	30	includes vendor CRUD, onboarding, and product aliases
`/api/auth`	27	register, verify, claim, password, login flows
`/api/admin/ura`	17	admin alias of URA services
`/api/ura`	17	primary URA app surface
`/api/superadmin`	12	vendor/supplier superadmin aliases
`/api/market`	10	staff, tokens, and market-related routes
`/api/admin/payments`	9	payment operations and monitoring

Route group	Count	Notes
`/api/suppliers`	7	supplier CRUD and setup-profile
`/api/admin/invoices`	6	invoice generation and retrieval
`/api/deliveries`	6	current delivery module
`/api/market-gates`	6	gate CRUD
`/api/marketLevels`	6	market level CRUD
`/api/markets`	6	market CRUD and lookup
`/api/products`	6	product alias routes
`/api/marketSections`	5	market section CRUD
`/api/shops`	5	shop CRUD

3. Collection Coverage

3.1 Main Postman collection vs backend code

Normalized comparison result:

? backend code: `191`

? Postman collection: `146`

? code-only routes: `45`

? collection-only routes: `0`

Interpretation:

? the collection is incomplete rather than misleading

? current backend dev added route surface that was never reflected into the collection

Largest missing categories:

1. EFRIS outbox routes
2. delivery routes
3. rent-payment PRN routes
4. admin URA aliases
5. superadmin alias routes

3.2 Smoke Hoppscotch vs backend code

Normalized comparison result:

? backend code: `191`

? smoke collection: `57`

? code-only routes: `134`

? collection-only routes: `0`

Interpretation:

? this file is behaving like a deliberate smoke subset, not a full backend contract

3.3 `Market Master Api_for_sabit.json`

Read:

? mixed utility/testing collection

? contains real-looking IDs and placeholder state

? not suitable as the canonical current API contract

3.4 URA provider-only Hoppscotch

Contains only:

? `POST /oauth2/token`

? `POST /partner-service/2.0.0/mda-services/get-tin-details`

? `POST /partner-service/2.0.0/prn-services/generate-prn`

Interpretation:

? useful for direct URA gateway testing

? not evidence of MMIS backend route parity

4. High-Value Missing Routes From Main Postman Collection

These are the highest-signal code routes missing from `Docs/MMIS_Backend_Postman_Collection.json`:

Method	Route	Backend file
`GET`	`/api/ura/efris-outbox`	`src/routes/ura.routes.js`
`POST`	`/api/ura/efris-outbox/enqueue-from-request`	`src/routes/ura.routes.js`
`POST`	`/api/ura/efris-outbox/:id/retry`	`src/routes/ura.routes.js`
`POST`	`/api/ura/prn/rent-payment`	`src/routes/ura.routes.js`
`POST`	`/api/ura/rent-payments/:invoicePaymentId/prn`	`src/routes/ura.routes.js`
`GET`	`/api/admin/ura/efris-outbox`	`src/routes/ura.routes.js`
`GET`	`/api/admin/ura/health`	`src/routes/ura.routes.js`
`POST`	`/api/admin/ura/prn/rent-payment`	`src/routes/ura.routes.js`
`GET`	`/api/deliveries`	`src/routes/delivery.routes.js`
`PATCH`	`/api/deliveries/:id/dispatch`	`src/routes/delivery.routes.js`
`PATCH`	`/api/deliveries/:id/receive`	`src/routes/delivery.routes.js`
`PATCH`	`/api/deliveries/:id/verify`	`src/routes/delivery.routes.js`

5. Inventory Verdict

Current backend dev has outgrown the main API collection. The collection is still useful, but it is no longer complete enough to serve as the only contract artifact for `dev`. The next canonical collection should be regenerated from current backend route inventory and explicitly include:

? URA admin aliases

? rent-payment PRN endpoints

? EFRIS outbox endpoints

? delivery module endpoints

? a decision on whether duplicate mount aliases remain supported or should be deprecated